

A Gentle Introduction to NP and NP-Completeness

Shangqi Lu, Yanlin Zhang

DSAA 2043, HKUST(GZ)

Motivation

Why Are Some Problems Hard?

- Many familiar problems have polynomial-time algorithms (shortest path, MST, BFS/DFS).
- Other similar-looking problems explode in complexity (longest path, Hamiltonian cycle).
- These often involve checking exponentially many possibilities.

We need a theory distinguishing polynomial-time solvable vs. intractable problems.

Combinatorial Explosion

- A route visiting n cities has $(n - 1)!$ possible tours.
- Exponential families: 2^n , $n!$, etc.
- Even small inputs make brute-force infeasible.

Polynomial vs. superpolynomial growth defines tractable vs. intractable.

P vs NP

Optimization vs Decision Problems

Many computational tasks come in two forms:

Optimization problem:

- Example: Find the *longest* simple path.
- Output is a structure (a path), not just yes/no.

Decision problem:

- Example: Does a simple path of length $\geq k$ exist?
- Output: YES or NO.

Why NP studies decision problems:

- Decision forms let us use verification algorithms and certificates.
- Optimization problems can almost always be converted to decision form.

Definition: A decision problem is in **P** if it can be decided in $O(n^k)$ time for some constant k .

- P contains polynomial-time decidable languages.
- Problems in P are “efficiently solvable”.
- Examples: BFS, DFS, Dijkstra, MST.

Verification Algorithms

A **verification algorithm** is a two-input algorithm $A(x, y)$:

- x : instance; y : certificate
- Certificate length is polynomial in $|x|$
- $A(x, y) = 1$ means y proves x is a YES instance

$$x \in L \iff \exists y \text{ such that } A(x, y) = 1.$$

Definition: A problem $L \in \mathbf{NP}$ iff there exists a polynomial-time verifier A and constant c such that:

$$x \in L \iff \exists y, |y| = O(|x|^c) \text{ and } A(x, y) = 1.$$

Interpretation: $\mathbf{NP}$ = problems with efficiently verifiable YES-solutions.

Certificate: A polynomial-length string enabling verification of a YES instance.

- For HAM-CYCLE: an ordered list of all vertices.
- For SAT: an assignment to all variables.

Hamiltonian Cycle

Hamiltonian Cycle (Decision Version)

Hamiltonian cycle of an undirected graph G is a simple cycle that contains each vertex in V .

Decision question: Does $G = (V, E)$ contain a simple cycle visiting every vertex exactly once?

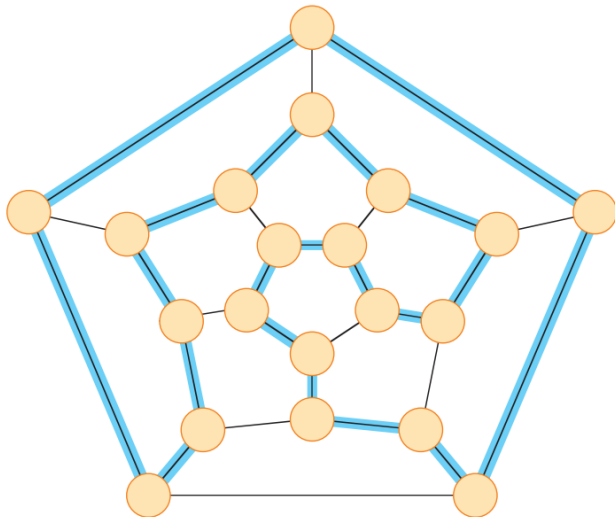
Formal definition:

$$HAM-CYCLE = \{\langle G \rangle : G \text{ has a Hamiltonian cycle}\}.$$

Interpretation:

- $\langle G \rangle$ is the binary encoding of graph G .
- The language contains all YES-instances.
- If $\langle G \rangle \in HAM-CYCLE$, a correct solver outputs YES.

Example



HAM-CYCLE $\in$ NP

Certificate $y = \langle v_1, \dots, v_n \rangle$:

- Check it lists all vertices exactly once.
- Check all edges (v_i, v_{i+1}) exist.
- Check edge (v_n, v_1) exists.

Runs in $O(n^2)$ time.

Thus HAM-CYCLE $\in$ NP.

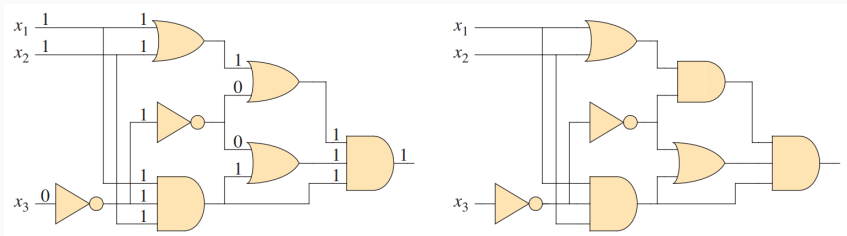
Circuit-SAT

Input: a boolean circuit C using AND/OR/NOT gates.

Decision question: Is there an assignment $(x_1, \dots, x_k)$ that makes the *output wire* evaluate to 1?

- The circuit is “satisfiable” if some input setting makes its output 1.
- Intuition: can we choose inputs so the “light turns on”?

Circuit-SAT Examples



Left: satisfiable. Right: unsatisfiable.

CIRCUIT-SAT $\in$ NP

Certificate: value of each wire.

- Check the certificate respects each gate's truth table.
- Check the final output wire equals 1.

Runs in polynomial time.

Thus CIRCUIT-SAT $\in$ NP.

NP-Complete

$L_1 \leq_P L_2$ iff there exists poly-time f such that:

$$x \in L_1 \iff f(x) \in L_2.$$

Interpretation: Solving L_2 helps solve L_1 .

NP-Complete Problems

A problem L is **NP-Complete** if:

- $L \in NP$
- Every problem $L' \in NP$ can be reduced to L in polynomial time

Interpretation: NP-Complete problems are the hardest problems in NP.

Putting the Classes Together

Key classes:

- **P**: solvable in polynomial time.
- **NP**: verifiable in polynomial time.
- **NP-Complete**: in NP and NP-hard.
- **NP-hard**: at least as hard as every problem in NP.

Relationships:

- $P \subseteq NP$.
- $NP\text{-Complete} \subseteq NP$.
- $NP\text{-hard} \supseteq NP\text{-Complete}$.
- **Open**: Is $P = NP$?

P, NP, NP-Complete: Side-by-Side

Class	Definition	Examples
P	Solvable in polynomial time	BFS, MST, Dijkstra
NP	Verifiable in polynomial time	HAM-CYCLE, SAT
NP-Complete	In NP and NP-hard	CIRCUIT-SAT, HAM-CYCLE

NP-Complete problems lie at the boundary of P vs NP.

Theorem: If any NP-Complete *problem* is in P, then $P = NP$.

- CIRCUIT-SAT is NP-complete.
- All NP-completeness proofs reduce from known NP-complete problems.

Why It Matters

- NP-complete problems unlikely have polynomial-time exact algorithms.
- Practical strategies:
 - Approximation algorithms
 - Heuristics / local search
 - Special-case polynomial-time algorithms
 - Randomized algorithms

- **P**: polynomial-time decidable.
- **NP**: polynomial-time verifiable with polynomial-size certificates.
- **NP-Complete**: hardest in NP.
- If any NP-Complete problem is in P, then $P = NP$.